Object Oriented Programming

Chapter 1:

Classes and Objects

Objectives

At the end of this Lecture, students will become familiar with:

- **Basic concepts of object-oriented programming;**
- **Defining classes;**
- **Creating objects from classes;**
- **Using Fields, Properties and Methods;**
- **Constructor Vs Destructor;**
- **Value-type Vs Reference-type.**

Outline

- Introduction
- Classes and Objects in C#
- Fields and Properties
- Constructor Vs Destructor
- Value-type Vs Reference-type
- Tutorial Example

Introduction

Object Oriented Programming (OOP) Vs Procedural Programing (PP)

- **PP (e.g., C, FORTRAN, Pascal, Basic, …)**
  - A program is divided into small parts called Procedures (or functions);
  - Each procedure processes data passed to it as input arguments, and may submit an obtained result to the procedure that called it;
  - PP reduces the number of code lines by avoiding code duplication;
  - PP is limited in terms of using only **predefined data types**;
  - PP does not support data hiding.
- **OOP (e.g., C++, Java, Python, C#, …)**
  - A program is divided into small parts called Objects;
  - Each object is an instance of a class which forms the data type of the object;
  - A class describes the contents of the objects that belong to it;
  - Class contents consist essentially of procedures (henceforth, called Methods) and fields;
  - OOP provides a clearer and more organized structure for the source code;
  - OOP supports data hiding;
  - Modifying an OOP program is easier than a PP program

Introduction

Basic Concepts of Object Oriented Programming

- **Encapsulation:** is the process of enclosing one or more items within a logical component to prevent access to implementation details. Some items of the software become hidden from other items that do not need it;
- **Abstraction:** It allows the developer to focus on the important things first and neglect the details to be dealt with when necessary; it allows making only relevant information visible. Encapsulation enables a developer to implement the desired level of abstraction
- **Inheritance:** It allows us to define a class in terms of another class, which makes it easier to create and maintain an application. This also provides an opportunity to reuse the code and speeds up implementation time.
- **Polymorphism:** It allows a method to have more than one implementation; and each implementation represents a different behavior. It provides an ability for the classes to implement different methods called through the same name..

Classes and Objects in C#

- In C#, a program consists of one or more classes;
- A class is a data structure that is used to describe a set of things having common properties and behaviors. These things are called **objects**.
- Each class contains, in addition to its data **fields**, a set of **methods** and **properties** that define the behavior of its objects and allow to access to its data and manipulate them;
- Class components are called **Class Members**.

```
public class Student {
     public  string name;
     public int Age { set; get; }
     public void PrintInfo(){
        Console.WriteLine("The name is: {0} , the age is: {1}", name, Age);
        }// end PrintInfo

      }// end class Student
```

Classes and Objects in C#

- Each **class member** is preceded by an **Access Specifier** to indicate the permissions to access it
  - The **public** specifier indicates that the member is public and can be accessed without restrictions
  - The **private** specifier indicates that a member is private and can only be accessed by members of the same class.
  - An access specifier can be declared for the whole class.
- Each **object** is an **instance** of a class, and it is created using the **new** keyword followed by a call of a **Class Constructor** which is a special method having the same name as the class.

```
Student obj1 = new Student();
obj1.Age = 24;
obj1.name = "MyName";
obj1.PrintInfo();
```

Fields and Properties

- **Fields** are variables that are defined in the class as one of its members, and when you create an object from the class it will have its own copy of the field values.
  - The field name begins conventionally with a lowercase letter, and declared as private.

- **Properties** are used to access fields from outside the class
  - The properties are similar to the fields in terms of usage
  - The properties are similar to the methods in terms of structure, but have no input parameters
  - Properties have **Accessors** that contain operations to be executed when their values are read or assigned new values
  - The property name begins conventionally with a uppercase letter, and declared as public.

Fields and Properties

- The property has often two **Accessors** : **get** and **set**,
  - The **get** accessor is used to read the value of the field corresponding to the property,
  - The **set** accessor is used to assign a value to the field corresponding to the property, the word "**value**" is used to denote the value.

```
private string name; // field name
public string Name { // property Name
    set { name = value;}
    get { return name; }
    }// end Name
```

- When the property is so simple that each accessor contains only one instruction, as in our case, the previous declaration of the **Name** property can be replaced by the following statement, and when this change is made to the **Student** class, the **name** field used outside the class must be replaced by the **Name** property.

```
private string name; // field name
public string Name { // property Name
      get => name;
      set => name = value;
        }// end Name
```

Fields and Properties

Auto_implemented properties

- Instead of declaring a field and a property together, an auto-implemented properties can be used:

```
public int Age { set; get; }
```

- During the compilation phase, the compiler will automatically create a field corresponding to the property and create the necessary **get** and **set** accessors;
- In most cases, Auto-Implemented Properties are used; Using fields is limited to define constants, static fields, or when there are conditions on the value to be assigned to the field;
- Accessors do not necessarily have the same access permissions, so the modification of the value of the field **name** corresponding to the **Name** property can be restricted to the class members:

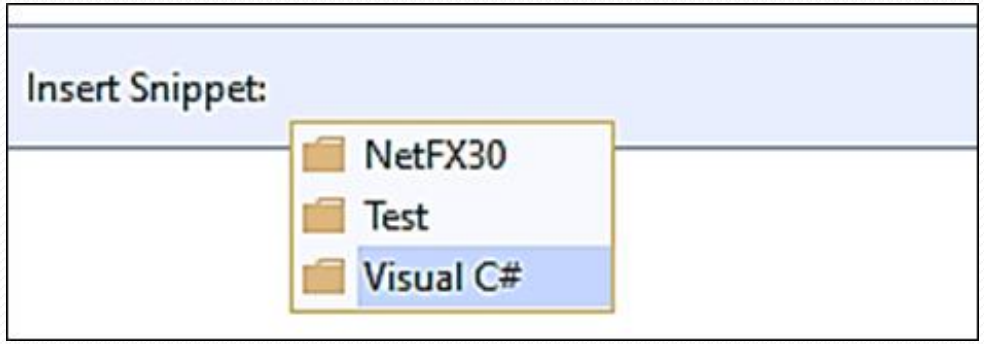
```
public int Age { private  set; get; }
```

Fields and Properties

- If the access specifier of a property is **private**, the access specifier of its accessors should not be **public**.
- Sometimes the property can have only **get** accessor (it is read only) and other instructions within the class must assign a value to the corresponding field. In this case, the field value is often assigned by the class constructor.

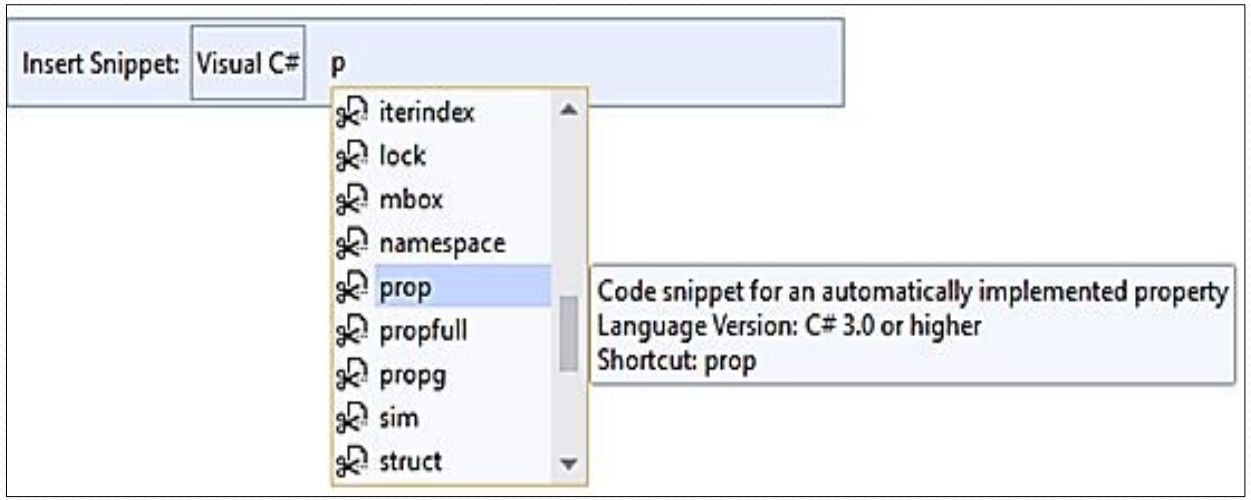
<u>Creating Auto_implemented properties using ready-made code snippets :</u>

- Press **Ctrl+k** then **Ctrl+x**, and then the following window appears:



Fields and Properties

- Choose **Visual C#**, then choose **prop** and the following window appears



- Press **Enter**, and the property is written as follows:

```
public int MyProperty { get; set; }
```

- Finally, make some modifications to get the desired property.

Constructor Vs Destructor

Constructor

```
Student obj1 = new Student();
```

- When creating an instance of a class (object), one of the class constructors is called
- The constructor allows to assign default values to the class members that need to be initialized
- The constructor is a special method having the same name as the class, it doesn't return any value and doesn't use **void** in its signature; usually, its access specifier is **public**.
- If no constructor is declared within the class, the compiler calls the **default constructor**, <u>which is a constructor that does not have input arguments</u>
- the **default constructor** initializes the fields/properties with default values corresponding to their data types. The value of (**0**) is assigned to numerical variables, **false** to logical variables, and **null** to reference variables.
- A class can have more than one constructor; and the constructors must differ in the number of input arguments and/or in their data types (overloading).
- If at least one constructor is defined, the compiler does not call the default constructor. In this case, if a default constructor is needed, it must be defined by the developer.

Constructor Vs Destructor

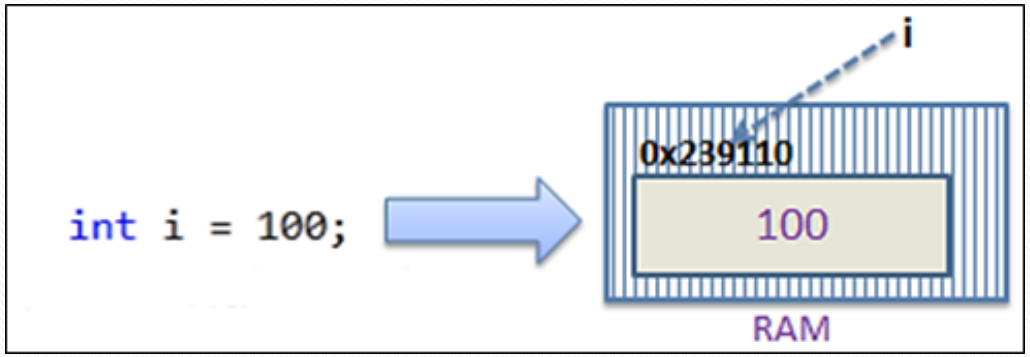
Destructor

- In C#, developers do not need to care about memory management; it is the mission of **Garbage Collector**;
- The **Garbage Collector** identifies the objects that have no reference (the program has finished using them) and recovers the memory allocated to them for using by other objects;
- For other resources such as databases, network connections, and files, the programmer must ensure that they are released when they are finished using them, and this can be done by defining a Destructor (sometimes called a Finalizer);
- The **Destructor** releases the reserved resources; the **Garbage Collector** automatically calls the **Destructor** just before restoring the memory assigned to the object;
- **Destructor** is a special method having the name of the class preceded by the character (~), and the instructions contained within it are executed just before the object is destroyed;
- The **Destructor** has no input arguments and therefore cannot be overloaded, it has no access specifier, and each class contains only one **Destructor**.

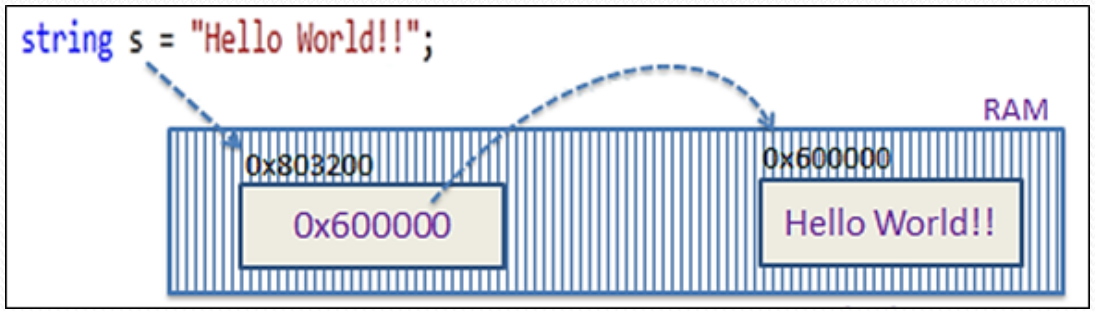
```
// destructor
        ~Student(){
            Console.WriteLine("The object will be destroyed ");


         }// end destructor
```

Value-type Vs Reference-type

- Value Types are simple, such as int, double, and float, and a variable from one of these patterns holds a specific value in the same memory location reserved for it.

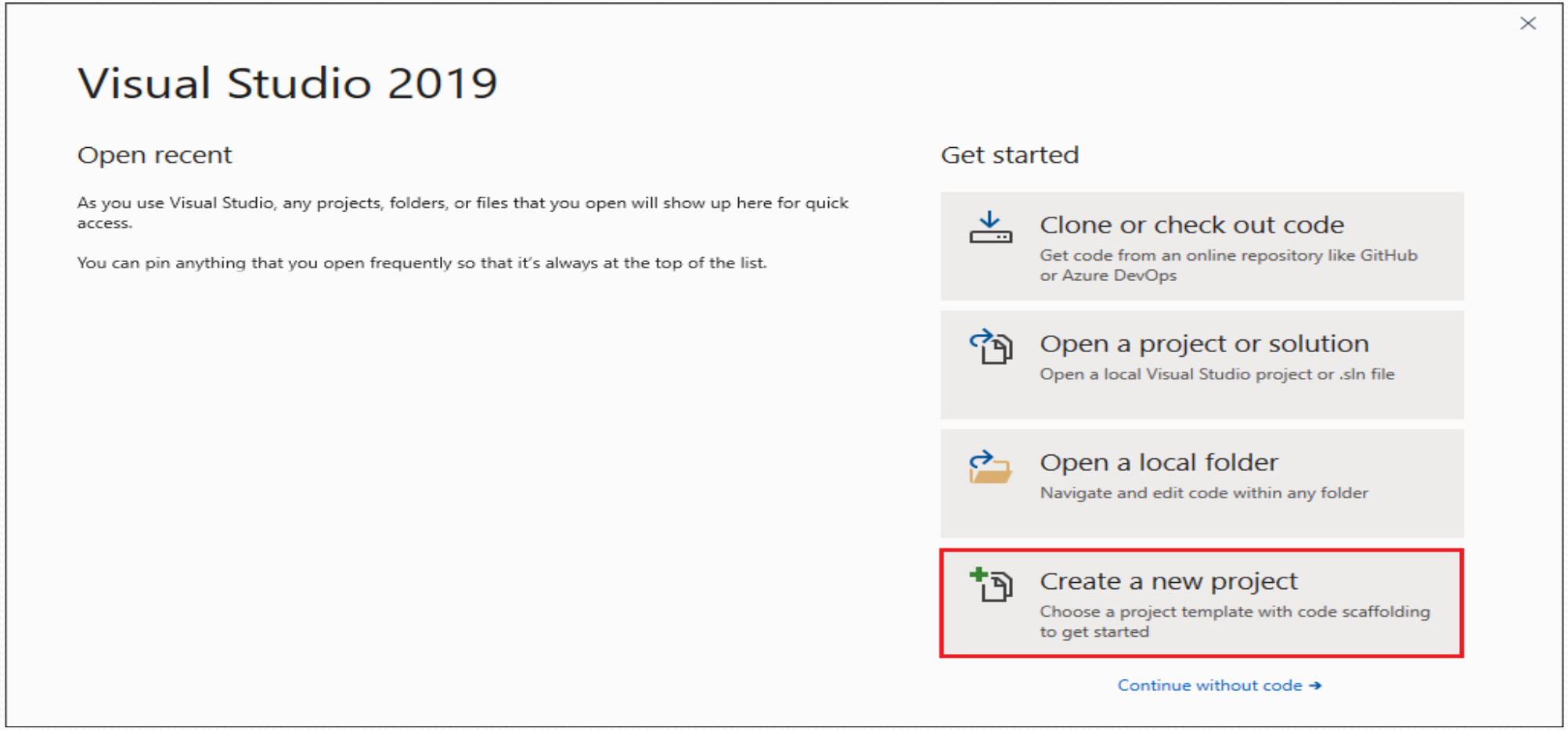


- Reference Types are complex, such as strings and arrays, and a variable from one of these types contains the address of a place in memory where the variable's value is stored.



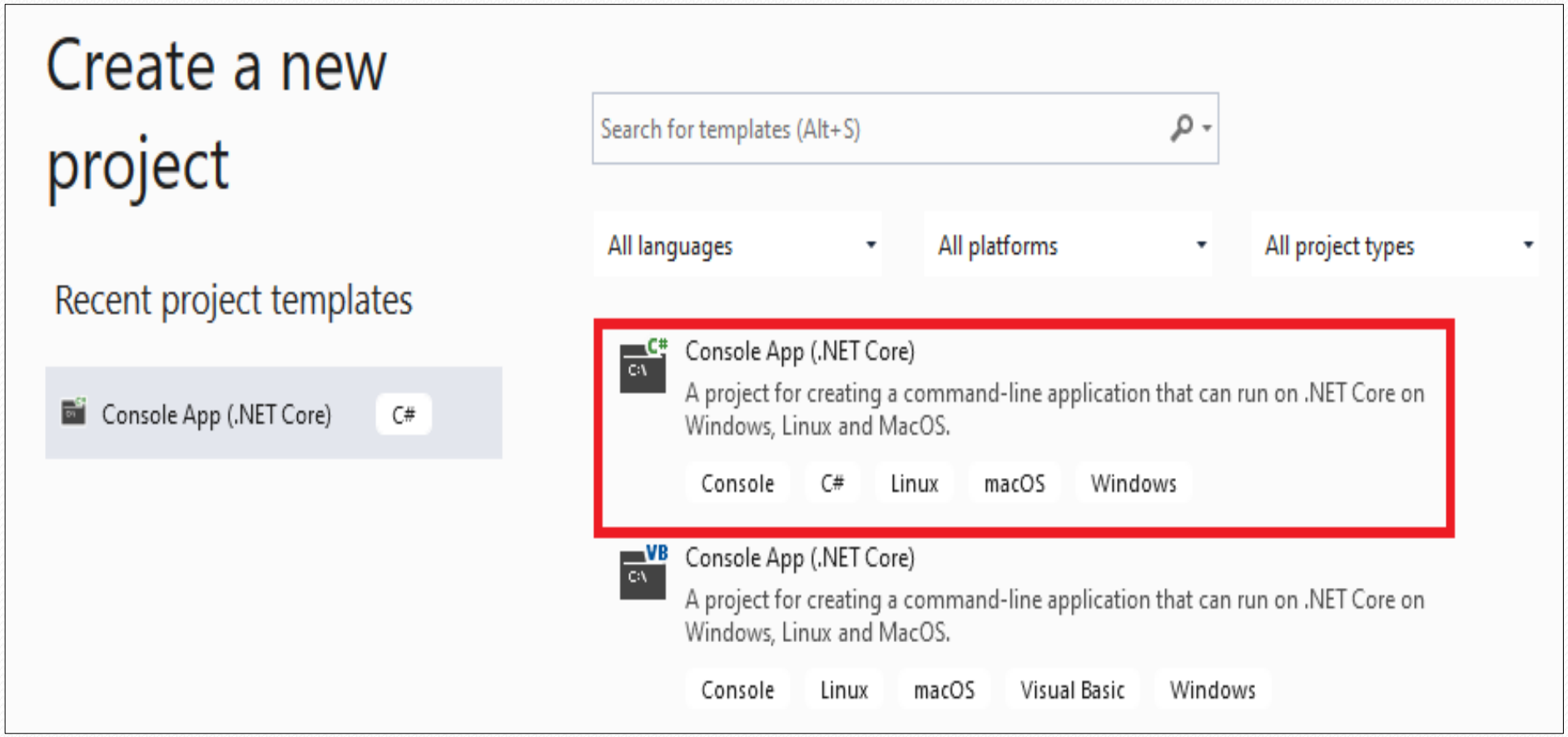
Tutorial Example

1. Launch Visual Studio 2019, then choose “Create a new project”



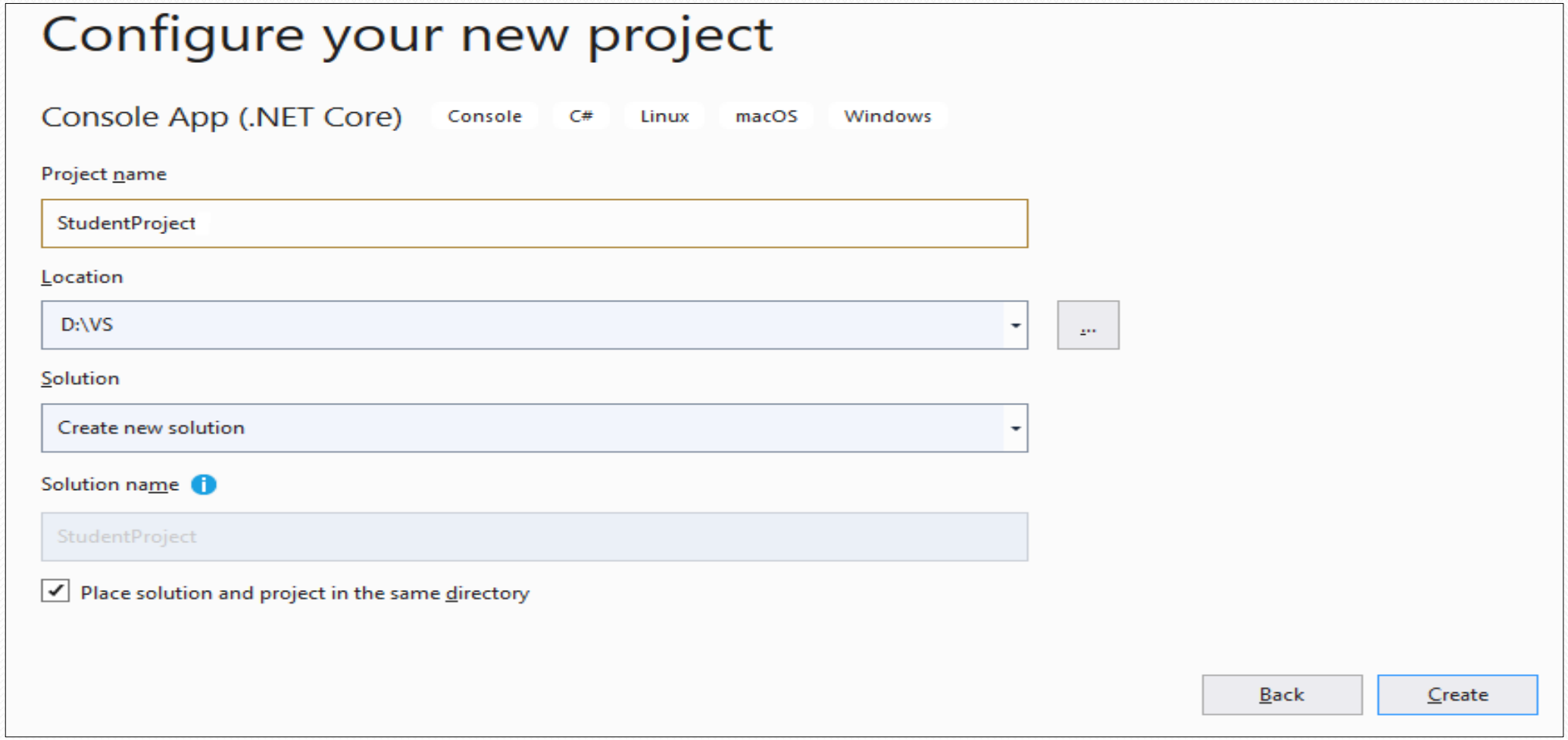
Tutorial Example

Choose the template **Console App (.NET Core)**

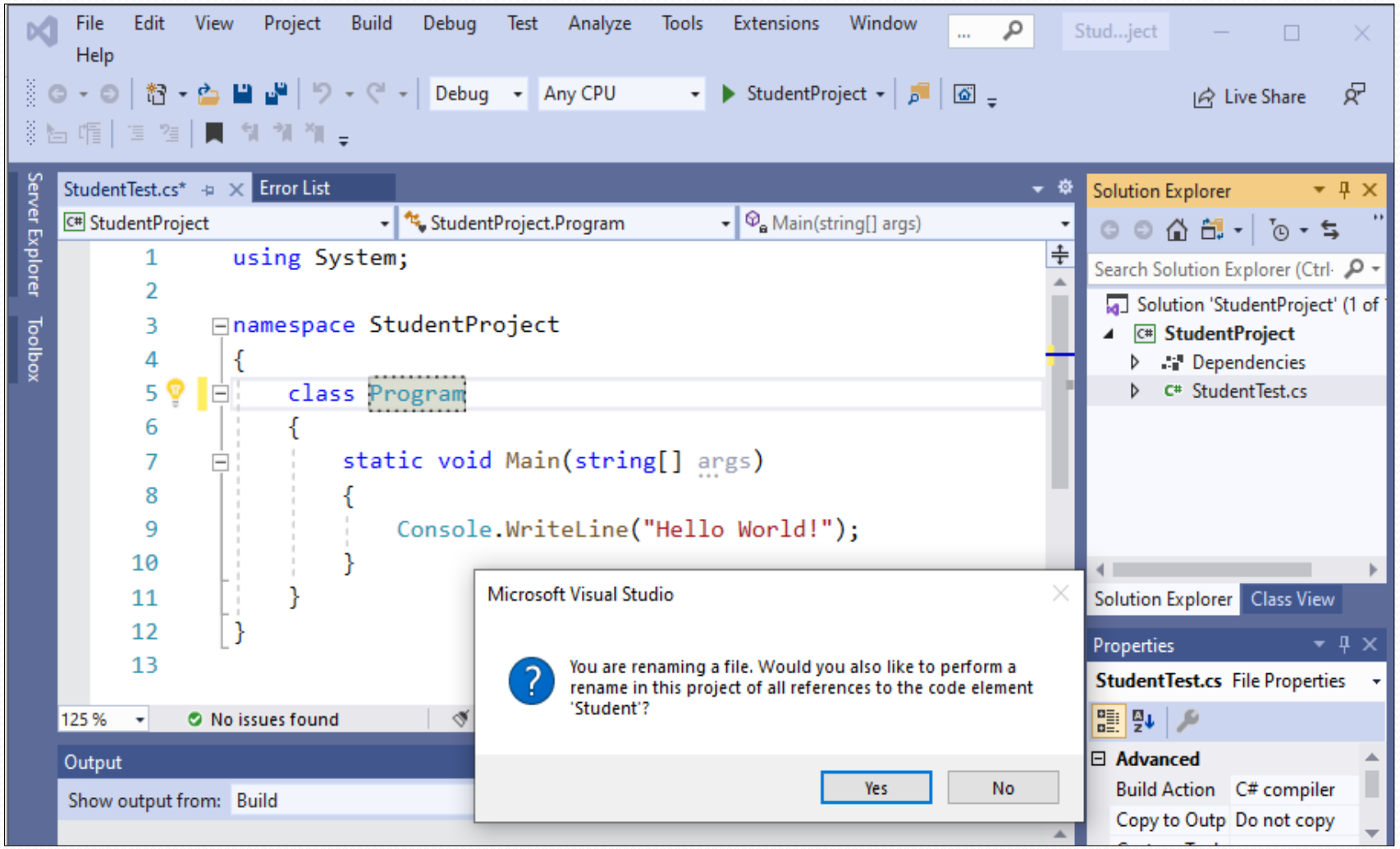


Tutorial Example

Enter the name of the project **StudentProject** and enter the location of the project **D: \ VS**, then press **Create**



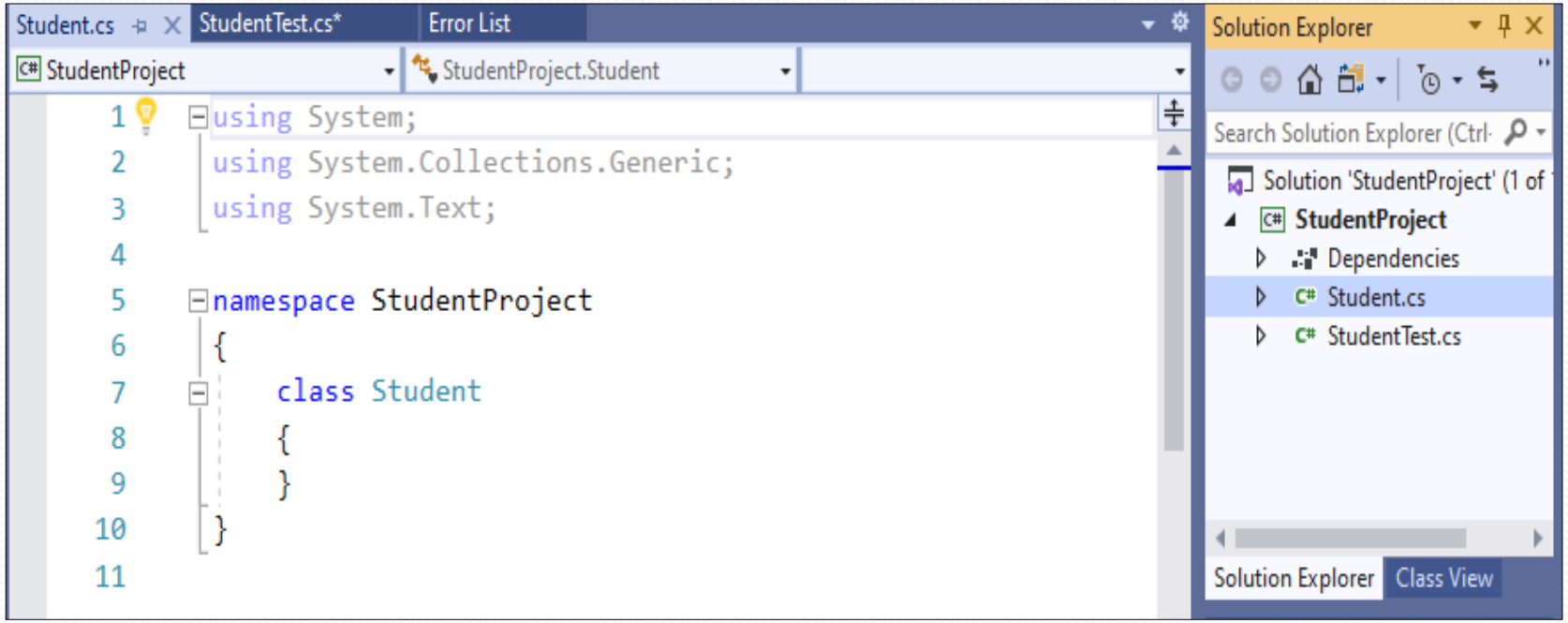
Tutorial Example



Replace the name of the text file with the name **StudentTest.cs**

Tutorial Example

- To add a new class, right-click on the **StudentProject** icon, a window appears, from which choose **add**, a new window appears, from which choose **Class**…
- Enter the name of the new class **Student** and then press **add**, a new file will be added with the name **Student.cs** in which the class **Student** is created



Tutorial Example

```
using System;
namespace StudentProject {
    class Student  {
        // name is a field
        private string name;

        // Name is a property
        public string Name {
            get => name;
            private set => name = value;
            // it is private, it means we cannot
            // change name from outside the class
    }// end Name
```

Modify the file **Student.cs** as follows:

```
// Age is an auto_implemented property,
        // the compiler will create the field age
        public int Age { set; get; }

        //constructor with two input parameters
        public Student(string stdName, int stdAge) {
            Name = stdName;
            Age = stdAge;
    }// end constructor
```

Tutorial Example

```
    // PrintInfo is a method for printing Name and Age

    public void PrintInfo( ) {
        Console.WriteLine("The student name is: {0}", Name);
        Console.WriteLine("The student age is: {0}", Age);
    }// end method PrintInfo
  }// end class Student

}//end namespace StudentProject
```

Tutorial Example

```
using System;
namespace StudentProject {
    class StudentTest   {
        static void Main( ) {
            //creating the object obj1 using constructor
            // with two input parameters
            Student obj1 = new Student("Salee", 18);

            //creating the object std using constructor
            //with one input parameter
            Student std = new Student("Hany", 20);
            //try to correct the name spelling
            // obj1.Name = "Saly";
            // Error CS0272: The property or indexer 'Student.Name' cannot be
  // used in this context because the set accessor is inaccessible
```

In the file **StudentTest.cs** write the code of the class **StudentTest** that is used to test the class **Student** and its members

Tutorial Example

```
        // print object information
        obj1.PrintInfo();
        std.PrintInfo();

        // waiting for a key press to prevent the screen
        // from closing quickly
        Console.ReadKey();
    }// end Main
  }// end  class StudentTest

}// end namespace StudentProject
```

Tutorial Example

- Note that we used the defined constructor to create the object **obj1** from the class **Student** with the values "Salee", 18 as input arguments and we used it again to create the object **std** with the values "Hany", 20 as input arguments. We also changed the name Salee to Saly for the first item, and used the PrintInfo method with both objects to print the information for each one.
- We tried to run the program and the following error message appeared:

```
The property or indexer 'Student.Name' cannot be used in this context because the set accessor is inaccessible
```

- The instruction **Obj1.Name = "Saly";** is the source of the error because the accesses specifier of the Name property is private, meaning that only members of its class can assign a value to the name field, and this explains why the constructor was able to assign a value to the field through the Name property.

Tutorial Example

After deleting the previous instruction and executing the previous code, we get the following output:

```
The student name is: Salee
The student age is: 18
The student name is: Hany
The student age is: 20
```

Summary

- The difference between procedural programming and object oriented programming;
- Declaring classes and creating objects from them;
- Understanding fields, properties and methods;
- The difference between constructor and destroctor;
- The difference between value-type and reference-type;
- Tutorial example.

Thank you very much for attending

You can kindly ask questions